02 · Handshake · arranque, Type 1 e envio de Type 2

IMPLEMENTADO · teste TCP separado :8082 · o browser não participa no HandshakeDemo

0 · Cada processo lê apenas o seu próprio .config

main() → HandshakeDemo.proxy(args) / HandshakeDemo.server(args)
CryptoConfigLoader.load(path, expectedRoot) → CryptoConfig + KeyMaterial + HandshakeConfig
CryptoConfigValidator.validate() consulta CipherSuite / MacSuite → ValidatedCryptoConfig
HandshakeConfigValidator.validate() → perfil executável AES-GCM; tudo antes de abrir sockets
Servidor: ServerSocket.accept(); proxy: Socket.connect(); ambos definem socket.soTimeout

PROXY · ProxyHandshake.perform(in, out, media)

Construtor volta a validar o perfil do handshake.
HandshakeWire.validateMedia(media)
SecureRandom: NP (clientRandom) e SEQX
DiffieHellmanSession.createSession(): par DH A/a
exportPublicKeyBytes(): A pública em formato DER
HandshakeWire.proposal(): ProposedCryptoConfig

SERVIDOR · ServerHandshake.perform(in, out)

Construtor volta a validar o perfil do handshake.
Obtém uma cópia local da PSK.
HandshakeWire.read(CLIENT_HELLO)
→ ProtocolMessageCodec.read(input, limite)
Bloqueia à espera da primeira mensagem.
Não usa ainda a chave DH recebida.

1 · Construir e autenticar ClientHello

ClientHelloPayload(SEQX, PCC, A, NP, tag vazia)
HandshakePayloadCodec.encodeClientHelloBody()
HandshakeWire.frame(): MessageHeader + payload
HandshakeKeySchedule.clientHelloAuthentication()
HandshakeCrypto.hmac(chave, header, body)
HandshakeWire.send() → ProtocolMessageCodec.write()

1 · Ler e validar ClientHello

HandshakeWire.read(): versão + Type esperado
HandshakePayloadCodec.decodeClientHello()
HandshakeKeySchedule.clientHelloAuthentication()
HandshakeCrypto.hmac() → MessageDigest.isEqual()
Compara PCC com HandshakeWire.proposal(local).
Valida SEQX; só depois faz trabalho DH.

TYPE 1 · header || SEQX || PCC || NP || A || HMAC-SHA256 →

Type 1: campos em claro; HMAC cobre header + body. A PSK nunca é enviada.

2 · Proxy aguarda ServerHello

HandshakeWire.read(SERVER_HELLO)
→ ProtocolMessageCodec.read()
A resposta tem um prefixo público NB.
NB permite derivar a chave antes de decifrar SRV.
O proxy ainda não conhece B nem o segredo Z.

2 · Preparar ServerHello

SecureRandom: NB (serverRandom) e SRV (desafio)
DiffieHellmanSession.createSession(): par DH B/b
calculateSecret(A): valida A e calcula Z = DH(b, A)
HandshakeCrypto.sha256(frame1) → H1
HandshakeKeySchedule.serverHelloProtection()
PSK + NP + NB + H1 → chave e IV de Type 2

3 · Cifrar e enviar Type 2

ServerHelloPayload(SRV, SEQX+1, CCC, B)
HandshakePayloadCodec.encodeServerHello()
HandshakeWire.frame() reserva o comprimento final.
HandshakeCrypto.encrypt(): AES-GCM + tag
AAD = header2 || NB || H1
HandshakeWire.send() → ProtocolMessageCodec.write()

← **TYPE 2 · header || NB || GCM(SEQY || SRV || CCC || B)**

NP e NB são públicos; SRV e B seguem cifrados. Cada frame inclui um header binário de 9 bytes.